SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Hoang-Nam Tran

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Performancevorhersage iteratorbasierter Datenbanksysteme mit Entscheidungsbäumen

Author: Hoang-Nam Tran
Examiner: Prof. Dr. Thomas Neumann
Supervisor: Maximilian Rieger, M.Sc.
Submission Date: 27.04.2025

I confirm that this bachelor’s thesis is my own work and I have documented all sources and material used.

Munich, 27.04.2025 Hoang-Nam Tran

Acknowledgments

Abstract

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Section 1
1.1.1 Subsection 1

2 Theoretical Background **3**
2.1 Databases and Query Execution 3
2.1.1 Iterator-Based Execution (PostgreSQL) 3
2.1.2 Compiled Query Execution (Umbra) 3
2.1.3 Pipelines and Pipeline Breakers 3
2.2 Database Performance Prediction 3
2.2.1 Benchmarking 3
2.2.2 Error Metrics 3
2.2.3 Traditional Cost Models and Learned Approaches 3
2.2.4 Neural Networks, Decision Trees, and T3 3
2.3 Related Work 3
2.4 ——————————— 3
2.5 Database Basics 3
2.6 Performance Prediction 3
2.7 Benchmarking 4
2.8 Cost Models 4
2.9 Decision Trees 4
2.10 PostgreSQL 4
2.11 Umbra 4
2.12 T3 4
2.13 Section 4
2.13.1 Subsection 4

3 Methodology **6**
3.1 Problem Statement and Prediction Target 6

3.2 Data and Preprocessing 6
3.3 Implementation 7
3.3.1 Mapping parsed PostgreSQL plans to the T3 plan format 7
3.3.2 Pipeline decomposition 8
3.3.3 PostgreSQL-native feature vectors 8
3.4 Training 11
3.4.1 Per-pipeline feature rows and per-tuple targets 11
3.4.2 Model and optimization 11
3.4.3 Data splits and reproducibility 12
3.5 Other Performance Prediction Implementations 12
3.6 Section 12
3.6.1 Subsection 12

4 Evaluation **14**
4.1 Ablation Study 14
4.1.1 PostgreSQL-native Features vs. Direct Transfer 14
4.1.2 Actual vs. Estimated Cardinalities 14
4.2 Section 14
4.2.1 Subsection 14

5 Future Work **16**
5.1 Section 16
5.1.1 Subsection 16

6 Conclusion **18**
6.1 Section 18
6.1.1 Subsection 18

Abbreviations **20**

List of Figures **21**

List of Tables **22**

Bibliography **23**

1 Introduction

1.1 Section

Citation test [Lam94].

Acronyms must be added in main.tex and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. \ac{TUM}, \ac{TUM}) Technical University of Munich (TUM), TUM

For more details, see the documentation of the acronym package[1].

1.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

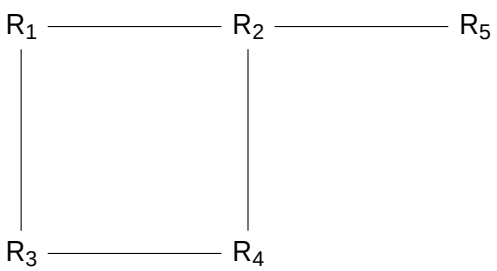

Figure 1.1: An example for a simple drawing.

[1] https://ctan.org/pkg/acronym

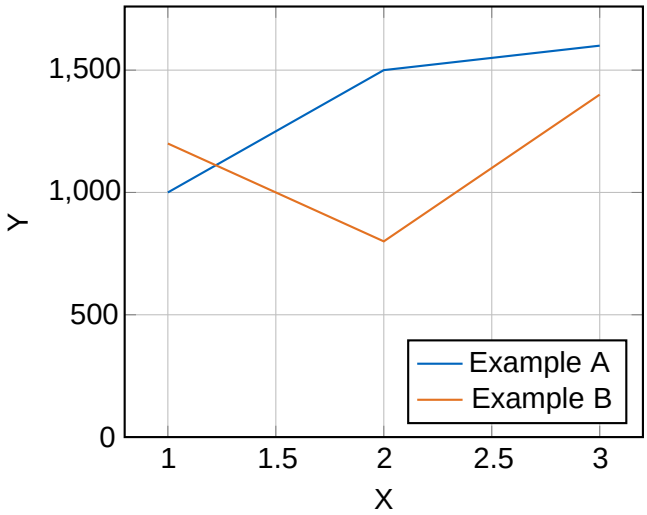

Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Theoretical Background

2.1 Databases and Query Execution

2.1.1 Iterator-Based Execution (PostgreSQL)

2.1.2 Compiled Query Execution (Umbra)

2.1.3 Pipelines and Pipeline Breakers

2.2 Database Performance Prediction

2.2.1 Benchmarking

sowas auch schreiben wie ssb, tpch, etc.

2.2.2 Error Metrics

2.2.3 Traditional Cost Models and Learned Approaches

2.2.4 Neural Networks, Decision Trees, and T3

2.3 Related Work

2.4 ————————————–

2.5 Database Basics

operator type, estimated/actual cardinalities, widths, and runtimes

2.6 Performance Prediction

draft: beschreiben, was performance prediction ist, was die ziele sind, etc. benchmarking, etc. Cost Model beschreiben

2.7 Benchmarking

timed queries, etc.

2.8 Cost Models

2.9 Decision Trees

2.10 PostgreSQL

2.11 Umbra

2.12 T3

2.13 Section

Citation test [Lam94].

Acronyms must be added in main.tex and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. \ac{TUM}, \ac{TUM}) TUM, TUM

For more details, see the documentation of the acronym package[1].

2.13.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 2.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] https://ctan.org/pkg/acronym

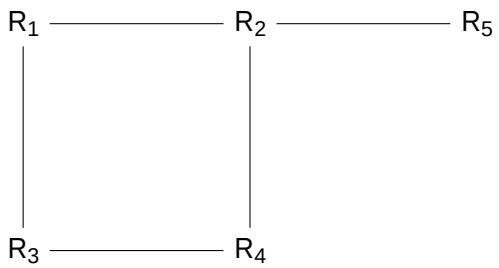

Figure 2.1: An example for a simple drawing.

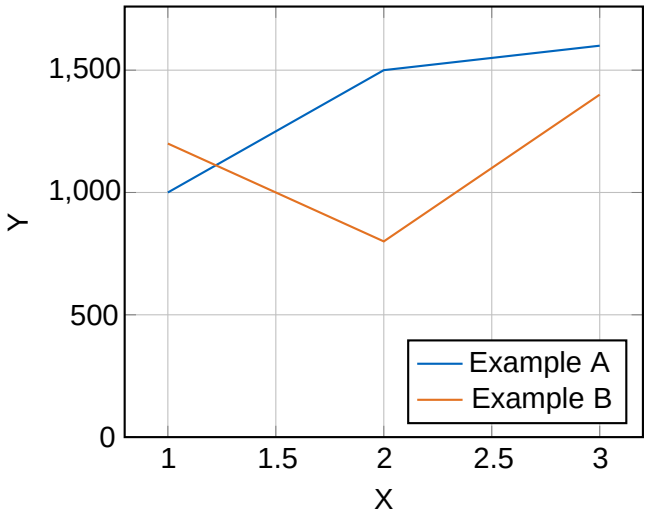

Figure 2.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 2.3: An example for a source code listing.

3 Methodology

3.1 Problem Statement and Prediction Target

In this thesis, we address the transfer of the T3 runtime prediction approach from Umbra to PostgreSQL parsed plans. The central methodological goal is to preserve T3's core modeling principles. Upon that, we adapt plan conversion and feature extraction to PostgreSQL-specific plan information.

Also, the prediction target is total query runtime. Following T3, each query plan is decomposed into pipelines and a per-tuple runtime is predicted for each pipeline. Afterwards, we aggregate the per-tuple and then the pipeline runtimes to obtain the final query-runtime estimate.

3.2 Data and Preprocessing

Regarding the training data, we use the public Zero-Shot benchmark data introduced with the Zero-Shot Cost Model. The benchmark is particularly suitable for this thesis because it consists of real-world datasets (e.g., IMDB) and established benchmark workloads (e.g., SSB, TPC-H and JOB). Overall, the benchmark contains 20 databases with large variation in tables, columns, and foreign-key relationships [HB22].

Having this multitude of databases allows us to test the generalization capabilities of our model. Similar to the T3 paper, we can train the model on all but one databases and test on the remaining one. Moreover, as the widely used JOB workload is included, we can compare our performance predictions to other implementations later on. Further, PostgreSQL is the intended database system of our implementation, which avoids the effort of building a separate PostgreSQL benchmark.

Following the benchmark construction described in the paper, queries are generated per database and executed on a PostgreSQL DBMS (v12) on c8220 nodes on the CloudLab platform. Then, they are stored as raw traces (i. e., the output of EXPLAIN ANALYZE). Next, the authors preprocess the queries to remove very long-running queries (queries above 30 s are excluded) to keep the benchmark practical and reproducible [HB22]. A further preprocessing step is to parse the raw traces into structured plan representations. These parsed plans (15000 per database) contain the operator nodes

found in the raw traces, but organize them in a tree structure with node-wise attributes. For every node, e.g., operator type, estimated/actual cardinalities, widths, and runtimes are stored. The tree structure can be used to create pipelines and the node information will be used to train the model. In the upcoming sections, we will describe these key implementation details.

Consequently, the parsed plans offer a good starting point for our pipeline and we can begin without further preprocessing.

3.3 Implementation

This section describes how a single parsed plan from the Zero-Shot benchmark is turned into the internal representation used by T3-style pipeline learning, and how we derive fixed-length feature vectors for each pipeline. The procedure follows the pipeline idea of T3 [RN25], but replaces Umbra-specific tuple-size and stage-based featurization with quantities that PostgreSQL exposes in EXPLAIN ANALYZE traces.

3.3.1 Mapping parsed PostgreSQL plans to the T3 plan format

Each training example starts from one element of the parsed_plans array in the benchmark JSON files. Such an object is a rooted tree: every node carries plan_parameters (operator name, estimated and actual cardinalities, tuple width, planner costs, optional filter predicates, timings, and further fields) and a list children of child subtrees.

We convert this tree recursively into the same nested operator structure that the original T3 implementation expects for Umbra plans [RN25]. PostgreSQL operator names are mapped to abstract operators (tablescan , join with a physical variant such as hash or index nested-loop, sort , groupby, unary pass-through nodes, and temporary operators for hash build and materialize). For joins, child order is normalized so that it matches T3's pipeline rules: for hash and merge joins the inner (build) subtree becomes the left child and the outer (probe) subtree the right child; nested-loop joins use the mapping required by the existing pipeline logic in the codebase. Each node receives stable integer identifiers (operatorId and analyzePlanId) used later for pipeline membership.

Widths instead of per-column information systems. Umbra plans attach detailed information about produced columns (information units, IUs) and their sizes. The parsed PostgreSQL plans do not provide an equivalent per-column breakdown; instead they report a single estimated tuple width per node (est_width). We therefore represent output size by one producedIUs entry per node whose estimatedSize is set from that

width (with a small minimum), rather than summing many per-IU sizes as in the Umbra path. Conceptually, this replaces fine-grained IU sizes with a single width feature per operator, which is what the benchmark offers.

Scan predicates in the parsed data appear as a nested filter_columns tree (logical and/or/not and comparison-like leaves). Where present, we translate this tree into the restriction format the T3 core understands (expression , input , optional direction), so that the same structural notion of a filter is available for any code path that still inspects restrictions.

To keep PostgreSQL-specific fields available after this normalization, every converted node additionally stores a copy of the original plan_parameters under a dedicated pg payload. Training and inference that use PostgreSQL-native features read only from this payload and from the pipeline layout, so we do not require a full database schema (table names in traces are not needed for that feature path; a minimal placeholder schema suffices for constructing QueryPlan objects in the implementation).

3.3.2 Pipeline decomposition

T3 groups operators into *pipelines*: sets of operators that can produce tuples in a pipelined fashion, separated by *pipeline breakers* such as sorting, grouping, or materialization [RN25]. We assign each analyzePlanId to a pipeline id by walking the converted tree with the same high-level rules as in the T3 work: sort and aggregate nodes break pipelines; a materialize node is treated as a breaker; hash and merge joins place the build side in the preceding pipeline and keep the join together with the probe side in the next pipeline.

For every pipeline we record the list of operator ids, and we derive start and stop times from the per-node actual timings in plan_parameters (converted to a common time unit) so that pipeline durations are available for training targets and bookkeeping. The total query runtime label is taken from plan_runtime when present.

3.3.3 PostgreSQL-native feature vectors

The original T3 feature mapper for Umbra builds a feature vector from *execution phases* within each pipeline (e.g. scan, build, probe phases) and from cardinalities and tuple sizes attached to those phases [RN25]. Relative to that design, our PostgreSQL adaptation differs in three main respects:

1. **PostgreSQL-native operator features.** Parsed PostgreSQL plans expose one EXPLAIN node per iterator operator; we aggregate features by *PostgreSQL operator name* (fixed layout per operator type) instead of by Umbra execution stage. This

matches the iterator-based plan tree and avoids inventing synthetic build/probe phases for every node.

2. **PostgreSQL planner cost estimates.** The pipeline-level block includes sums and extrema of PostgreSQL's est_cost and est_startup_cost from plan_parameters. These are optimizer-internal estimates of work and startup cost, not observed runtimes; they are absent from the Umbra-oriented feature path in the original T3 code.

3. **Planned parallelism.** We sum workers_planned over all operators in the pipeline so the model can see how much intra-node parallelism the PostgreSQL planner expected (zero on purely sequential plans).

An excerpt of the resulting fixed-length vector (pipeline-level aggregates plus a slice of the per–operator-type block) is given in Figure 3.1.

Concretely, we enumerate all operator types that occur in the parsed Zero-Shot plans (scans, bitmap scans, joins, hash and materialize build nodes, sort, aggregate variants, gather nodes, and others) and define a fixed feature layout. For each pipeline the vector consists of two concatenated blocks.

In the Umbra implementation of T3, one pipeline row is obtained by summing feature vectors indexed by operator type and execution phase (scan, build, probe, . . .); pipeline scale is reflected inside those per-phase rows [RN25]. PostgreSQL EXPLAIN nodes do not carry Umbra phases, so a single summed grid of that form is unavailable. The **pipeline-level** block therefore collects these quantities as explicit pipeline-wide statistics: summed and extremal planner costs, total planned workers, mean tuple width, and aggregates over scan predicates and filter structure. The **operator-type** block then supplies fixed entries per PostgreSQL operator name (counts, cardinalities, percentages), playing a role comparable to summing type-specific phase contributions in the original design.

1. **Pipeline-level aggregates.** Over all operators in the pipeline we compute sums and extrema of actual cardinalities (falling back to estimates where actual values are missing), sums of PostgreSQL planner costs (est_cost) and startup costs (est_startup_cost), the sum of planned parallel workers (workers_planned), the average estimated tuple width, counts of operators by coarse class (scans, joins, sorts, aggregates, temporaries, pass-through), scan-specific sums and indicators (including whether scans have filters), counts of filter constructs (e.g. logical and/or, comparisons, LIKE, IN) derived from filter_columns, the number of operators in the pipeline, and the root operator's output cardinality as a query-global signal.

2. **Operator-type block.** For each PostgreSQL operator type in a fixed ordering, we accumulate a constant (occurrence count) and a small set of numeric descriptors (for example aggregated input and output cardinalities and widths, fractions relative to total scan cardinality in the pipeline, and expression-type fractions for scans), using definitions that match the join semantics of the converted tree (e.g. which child counts as probe stream versus build side for hash joins).

Pipeline-level block (excerpt; first 9 of 24 features)	
`pg_act_card_sum`	6541.0
`pg_act_card_max`	6537.0
`pg_est_cost_sum`	16882.95
`pg_est_cost_max`	16877.3
`pg_est_startup_sum`	125.82
`pg_workers_planned_sum`	4.0
`pg_est_width_avg`	10.0
`pg_num_scan`	1.0
`pg_num_join`	1.0
Operator-type block (excerpt; Nested_Loop slice; 130 entries in full)	
`Nested_Loop_const`	1.0
`Nested_Loop_in_percentage`	0.11473152822395594
`Nested_Loop_right_card`	98378.0
`Nested_Loop_out_percentage`	0.008260670032124828
(remaining operator-type entries omitted)	

Figure 3.1: Excerpts of the pipeline-level block (first nine features) and of the Nested_Loop entries in the operator-type block, for one pipeline from query index_workload_100k_s2_c8220_890on the TPC-H holdout test split (15 000 queries in that run). The full concatenated vector has 24+ 130 = 154 dimensions; remaining pipeline and operator entries are omitted.

Figure 3.1 illustrates how the two blocks are laid out end-to-end: pipeline-level entries summarize the whole pipeline (cardinalities, planner costs, parallelism, filters, . . .), while the operator block repeats the same layout for every PostgreSQL operator type, filling in only the entries that correspond to operators present in that pipeline.

The layout is fixed so that every pipeline maps to one row of a feature matrix independent of plan size; operators that never appear contribute zeros in their entries.

In summary, we preserve T3's pipeline decomposition and per-pipeline prediction target [RN25], while replacing Umbra-oriented tuple and stage features with a width-based size model and with PostgreSQL planner and cardinality signals derived from the parsed plan trees.

3.4 Training

This section describes how we fit the predictor to the Zero-Shot parsed plans. We follow the tuple-centric training target proposed in T3 [RN25], but use the PostgreSQL-native feature matrix and the train–test splits implemented in our zeroshot training scripts.

3.4.1 Per-pipeline feature rows and per-tuple targets

Each training example contributes one row per pipeline: the feature vector is the corresponding row of the pipeline matrix from Section 3.3 (one fixed-length vector per pipeline). The label for that row is not the full pipeline duration directly, but the *per-tuple pipeline time*, i.e. the observed pipeline duration divided by the pipeline's scan cardinality [RN25]. Intuitively, the model learns how much time the pipeline spends per tuple entering from the scan side, which stabilizes learning across queries with very different result sizes.

Concretely, the scan cardinality of a pipeline is the sum of the cardinalities of all scan operators belonging to that pipeline (actual cardinalities from the parsed plan, with estimates as a fallback where needed). At inference time, the model predicts a value for each pipeline row; that prediction is mapped back to a pipeline duration by multiplying with the same scan cardinality, and predicted query runtime is the sum of the predicted pipeline times – again as in T3 [RN25].

Training rows with an all-zero feature vector are dropped, because they carry no usable signal.

3.4.2 Model and optimization

We use gradient boosted decision trees as implemented in LightGBM [Ke+17], in line with the original T3 work [RN25]. The booster is trained with a mean absolute percentage error (MAPE) objective on the training rows. As in the reference implementation bundled with this project, each per-tuple target t is clamped to a small positive floor, transformed as $\log(t)$, and passed to LightGBM; predictions are mapped back with the exponential of the negative raw score before multiplication with scan cardinalities.

Following T3 [RN25], we cap the number of boosting rounds at **200** trees. The motivation is the same as in the paper: a relatively shallow ensemble limits capacity and helps avoid overfitting to idiosyncrasies of the training workloads while remaining expressive enough for accurate runtime ranking and estimation.

3.4.3 Data splits and reproducibility

Generalization across benchmarks. For the main cross-benchmark evaluation, we use a leave-one-benchmark-out split on the parsed-plan JSON files: all files whose path belongs to benchmark directories other than the held-out one form the training pool, and all files from the held-out benchmark form the test set. This matches the intent of the Zero-Shot benchmark [HB22] and of T3's cross-instance evaluation [RN25], namely to measure whether the model transfers to an unseen database or workload folder.

Train and validation within the training pool. Inside the training pool, complete queries (entire plans) are partitioned at random into a training fraction of 80 % and a validation fraction of 20 %. The random seed for this split defaults to **42** in the implementation.

Environment. Model training and evaluation for the results reported in this thesis were run on a personal laptop: Apple MacBook Pro (16-inch, 2021) with Apple M1 Pro (10 CPU cores, 16 GPU cores), 16 GB unified memory, and 512 GB SSD, under macOS 15.7.3.

3.5 Other Performance Prediction Implementations

Add other implementations that you want to compare to.

3.6 Section

Citation test [Lam94].

Acronyms must be added in main.tex and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. \ac{TUM}, \ac{TUM}) TUM, TUM

For more details, see the documentation of the acronym package[1].

3.6.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

[1] https://ctan.org/pkg/acronym

Table 3.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

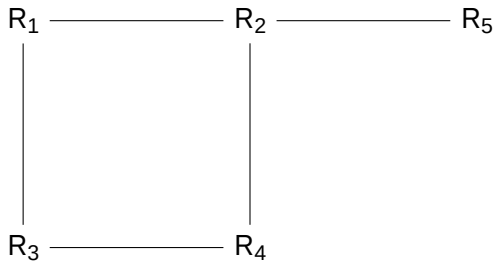

Figure 3.2: An example for a simple drawing.

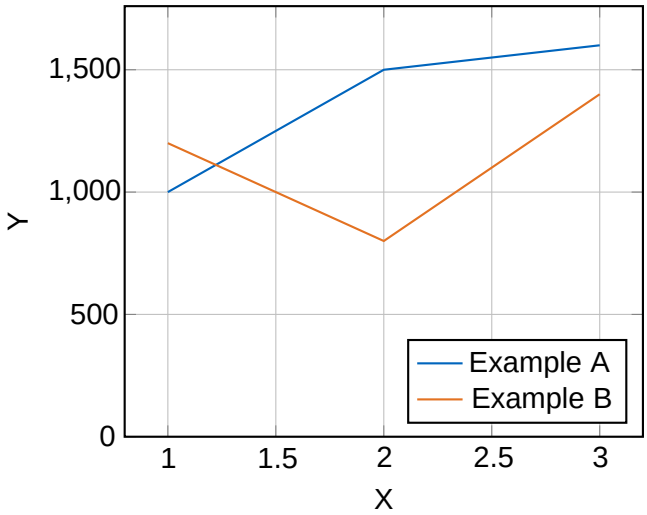

Figure 3.3: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 3.4: An example for a source code listing.

4 Evaluation

4.1 Ablation Study

4.1.1 PostgreSQL-native Features vs. Direct Transfer

4.1.2 Actual vs. Estimated Cardinalities

4.2 Section

Citation test [Lam94].

Acronyms must be added in main.tex and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. \ac{TUM}, \ac{TUM}) TUM, TUM

For more details, see the documentation of the acronym package[1].

4.2.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 4.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] https://ctan.org/pkg/acronym

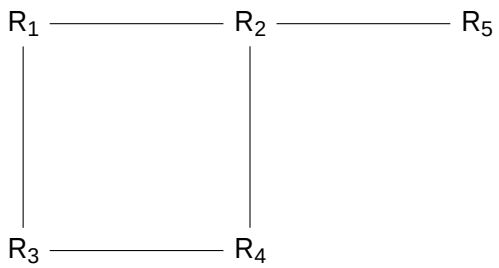

Figure 4.1: An example for a simple drawing.

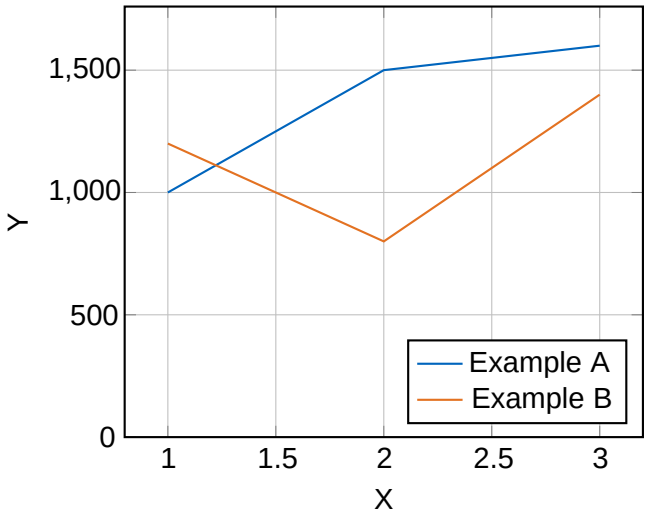

Figure 4.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 4.3: An example for a source code listing.

5 Future Work

Data from Zeroshot kann limitierender faktor sein.

5.1 Section

Citation test [Lam94].

Acronyms must be added in main.tex and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. \ac{TUM}, \ac{TUM}) TUM, TUM

For more details, see the documentation of the acronym package[1].

5.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 5.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] https://ctan.org/pkg/acronym

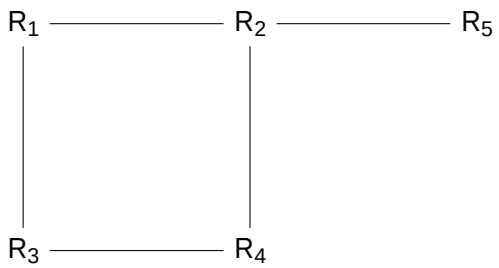

Figure 5.1: An example for a simple drawing.

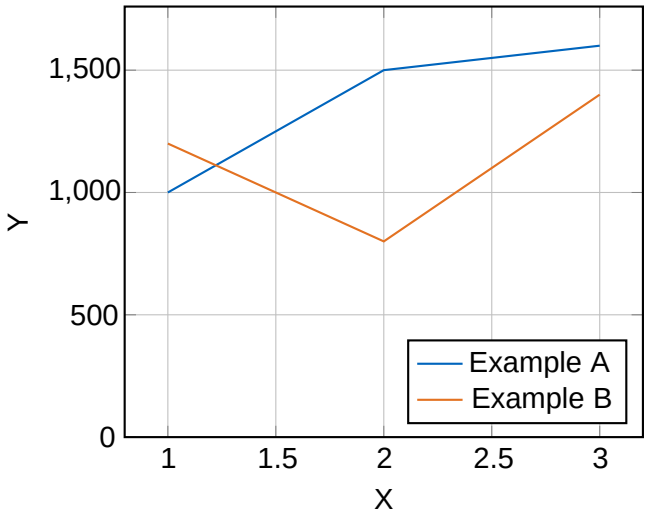

Figure 5.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 5.3: An example for a source code listing.

6 Conclusion

6.1 Section

Citation test [Lam94].

Acronyms must be added in main.tex and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. \ac{TUM}, \ac{TUM}) TUM, TUM

For more details, see the documentation of the acronym package[1].

6.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 6.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

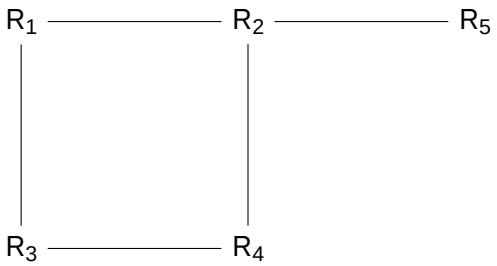

Figure 6.1: An example for a simple drawing.

[1] https://ctan.org/pkg/acronym

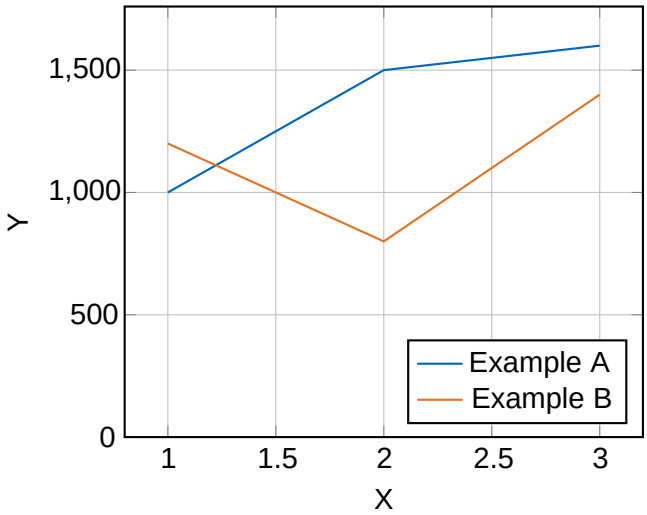

Figure 6.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 6.3: An example for a source code listing.

Abbreviations

TUM Technical University of Munich

Bibliography

[HB22]